Sign in

Sheetalshee / WDM_EXP5 Public

forked from dharshanpt/WDM_EXP5

Code Pull requests Actions Projects Security Insights

main Go to file Code

This branch is 4 commits ahead of `dharshanpt/WDM_EXP5:main`.

Sheetalshee Update README.md b7b66aa · 2 minutes ago

.gitignore	Initial commit	2 years ago
LICENSE	Initial commit	2 years ago
README.md	Update README.md	2 minutes ago

README License

EX5 : Information Retrieval Using Boolean Model in Python

NAME: SHEETAL R

DATE:

AIM:

To implement Information Retrieval Using Boolean Model in Python.

Description:

The Boolean model in Information Retrieval (IR) is a fundamental model used for searching and retrieving information from a collection of documents. It operates on the principles of set theory and logic, where documents are represented as sets of terms or words, and queries are expressed as Boolean expressions using logical operators such as AND, OR, and NOT.

Procedure:

1. ***Initialize the BooleanRetrieval class:*** The BooleanRetrieval class is defined to manage the indexing and searching of documents.
2. ***Constructor and Index Initialization:*** The class constructor initializes an empty index to store the inverted index mapping terms to documents.
3. ***Indexing Documents:***

   a) The index_document method is responsible for indexing documents.

   b) Tokenize the text content of documents, converting them into lowercase terms.

   c) For each term in the document, it adds an entry in the index, associating the term with the document ID.

4. ***Fetch Web Page Text:***

   a) The fetch_webpage_text method uses the requests library to fetch content from a given URL.

   b) Extract text content from the fetched HTML using BeautifulSoup.

   c) The extracted text is returned for further processing.

5. ***Boolean Search:***

   a) The boolean_search method performs Boolean searches on the indexed documents.

   b) Tokenize the input query and iterates through its terms.

   c) For each term in the query, it retrieves documents containing that term and performs Boolean operations (AND, OR, NOT) based on the query's structure.

Program:

```
import numpy as np
import pandas as pd

class BooleanRetrieval:
    def __init__(self):
        self.index = {}
        self.documents_matrix = None

    def index_document(self, doc_id, text):
        terms = text.lower().split()
        print("Document -", doc_id, terms)
```

```
        for term in terms:
            if term not in self.index:
                self.index[term] = set()
            self.index[term].add(doc_id)

    def create_documents_matrix(self, documents):
        terms = list(self.index.keys())
        num_docs = len(documents)
        num_terms = len(terms)

        self.documents_matrix = np.zeros((num_docs, num_terms), dtype=int)

        for i, (doc_id, text) in enumerate(documents.items()):
            doc_terms = text.lower().split()
            for term in doc_terms:
                if term in self.index:
                    term_id = terms.index(term)
                    self.documents_matrix[i, term_id] = 1

    def print_documents_matrix_table(self):
        df = pd.DataFrame(self.documents_matrix, columns=self.index.keys())
        print(df)

    def print_all_terms(self):
        print("All terms in the documents:")
        print(list(self.index.keys()))

    def boolean_search(self, query):
        tokens = query.lower().split()
        result_set = set()

        i = 0
        while i < len(tokens):
            token = tokens[i]

            if token == "and":
                i += 1
                term_docs = self.index.get(tokens[i], set())
                result_set = result_set.intersection(term_docs)

            elif token == "or":
                i += 1
                term_docs = self.index.get(tokens[i], set())
                result_set = result_set.union(term_docs)

            elif token == "not":
                i += 1
                term_docs = self.index.get(tokens[i], set())
                result_set = result_set.difference(term_docs)
```

```python
            else:
                # first term or standalone
                term_docs = self.index.get(token, set())
                if not result_set:
                    result_set = term_docs.copy()
                else:
                    # default operator AND
                    result_set = result_set.intersection(term_docs)
            i += 1

        return list(result_set) if result_set else []


if __name__ == "__main__":
    indexer = BooleanRetrieval()

    documents = {
        1: "Python is a programming language",
        2: "Information retrieval deals with finding information",
        3: "Boolean models are used in information retrieval"
    }

    for doc_id, text in documents.items():
        indexer.index_document(doc_id, text)

    indexer.create_documents_matrix(documents)
    indexer.print_documents_matrix_table()
    indexer.print_all_terms()

    query1 = input("Enter your boolean query (use AND / OR / NOT): ")
    results = indexer.boolean_search(query1)
    if results:
        print(f"Results for '{query1}': {results}")
    else:
        print("No results found for the query.")
```

Output:

AND:

```
Document - 1 ['python', 'is', 'a', 'programming', 'language']
Document - 2 ['information', 'retrieval', 'deals', 'with', 'finding', 'information']
Document - 3 ['boolean', 'models', 'are', 'used', 'in', 'information', 'retrieval']
   python  is  a  programming  language  information  retrieval  deals  with  \
0       1   1  1           1         1            0          0      0     0
1       0   0  0           0         0            1          1      1     1
2       0   0  0           0         0            1          1      0     0

   finding  boolean  models  are  used  in
0        0        0       0    0     0   0
1        1        0       0    0     0   0
2        0        1       1    1     1   1
All terms in the documents:
['python', 'is', 'a', 'programming', 'language', 'information', 'retrieval', 'deals', 'with', 'finding', 'boolean', '
Enter your boolean query (use AND / OR / NOT): information and retrieval
Results for 'information and retrieval': [2, 3]
```

OR:

```
Document - 1 ['python', 'is', 'a', 'programming', 'language']
Document - 2 ['information', 'retrieval', 'deals', 'with', 'finding', 'information']
Document - 3 ['boolean', 'models', 'are', 'used', 'in', 'information', 'retrieval']
   python  is  a  programming  language  information  retrieval  deals  with  \
0       1   1  1            1         1            0          0      0     0
1       0   0  0            0         0            1          1      1     1
2       0   0  0            0         0            1          1      0     0

   finding  boolean  models  are  used  in
0        0        0       0    0     0   0
1        1        0       0    0     0   0
2        0        1       1    1     1   1
All terms in the documents:
['python', 'is', 'a', 'programming', 'language', 'information', 'retrieval', 'deals', 'with', 'finding', 'boolean', 'm
Enter your boolean query (use AND / OR / NOT): python or information
Results for 'python or information': [1, 2, 3]
```

NOT:

```
Document - 1 ['python', 'is', 'a', 'programming', 'language']
Document - 2 ['information', 'retrieval', 'deals', 'with', 'finding', 'information']
Document - 3 ['boolean', 'models', 'are', 'used', 'in', 'information', 'retrieval']
   python  is  a  programming  language  information  retrieval  deals  with  \
0       1   1  1            1         1            0          0      0     0
1       0   0  0            0         0            1          1      1     1
2       0   0  0            0         0            1          1      0     0

   finding  boolean  models  are  used  in
0        0        0       0    0     0   0
1        1        0       0    0     0   0
2        0        1       1    1     1   1
All terms in the documents:
['python', 'is', 'a', 'programming', 'language', 'information', 'retrieval', 'deals', 'with', 'finding', 'boolean',
Enter your boolean query (use AND / OR / NOT): python not deals
Results for 'python not deals': [1]
```

Result:

Thus, the python program to implement Information Retrieval Using Boolean Model is executed successfully.

Releases

No releases published

Packages

No packages published

Contributors

No contributors

Terms Privacy Security Status Community Docs Contact Manage cookies
Do not share my personal information

© 2026 GitHub, Inc.